

Assignment lab 1
Gang Ferdinand Dinga
Date: 28/10/2025

W1D1

Learn and Repeat

Pattern 1. “for all” and “there exists” pattern

Given an array A, if all elements of A are positive, return 1. Else return 0.

Algorithm 1. (Counting method)

```
count = 0;
for (i = 0; i < n; i++)
    if (A[i] > 0) count++;
(count == n)? true : false;
```

check the condition. If true, increment the counter. If the counter value is n, return true. Else return false.

Algorithm 2. (Test the opposite condition method)

```
for (i = 0; i < n; i++)
    if (A[i] <= 0) return false;
return true;
```

check the “opposite condition”. If true, return false. Return true outside the loop.

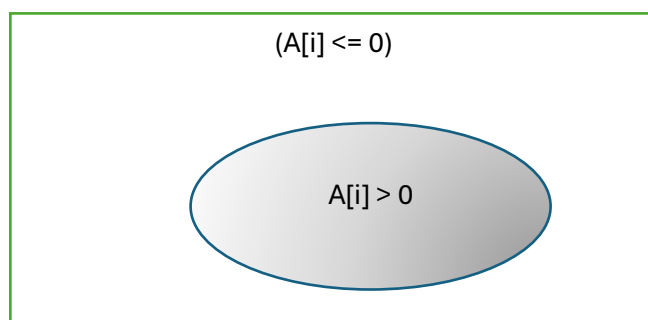
	Best case	Average case	Worst case
Algorithm 1	$O(n)$	$O(n)$	$O(n)$
Algorithm 2	$O(1)$	$O(n)$	$O(n)$

Lower bound of the problem

What is the minimum information required to conclude the problem statement in the affirmative? (This a very confusing idea for students with less exposure to logic.)

You need to check all elements of A. Hence the lower bound of the problem is $\Omega(n)$.

Since the lower bound of the problem is equal to the **worst case time complexity** in Solution 1 (Solution 2), Solution 1 (Solution 2) is optimal.



Exercise 1

An array is defined to be a **235 array** if the number of elements divisible by 2 plus the number of elements divisible by 3 plus the number of elements divisible by 5 plus the number of elements not divisible by 2, 3, or 5 is equal to the number of elements of the array.

Write a method named `is123Array` that returns 1 if its array argument is a 235 array, otherwise it returns 0.

Note : A number can be divisible by more than one number. For example, 10 is divisible by both 2 and 5.

Algorithm 3. (Counting method)

```
public static int is123Array (int[] arr)
{
    int total =0;
    for(int num :arr)
    {
        total += (num%2 ==0?1:0) + (num%3 ==0?1:0) + (num%5 ==0?1:0) + ((num%2 !=0 &&
            num%3 !=0 && num%5 !=0)?1:0);
    }
    return (total == arr.length)?1:0;
}
```

Algorithm 4. (Test the opposite condition method)

```
public static int is123ArrayOpposite(int[] arr) {
    int total = 0;
    for (int num : arr) {
        total += (num % 2 == 0 ? 1 : 0)
            + (num % 3 == 0 ? 1 : 0)
            + (num % 5 == 0 ? 1 : 0)
            + ((num % 2 != 0 && num % 3 != 0 && num % 5 != 0) ? 1 : 0);
    }
    if (total != arr.length) return 0; // opposite condition
    return 1;
}
```

	Best case	Average case	Worst case
--	-----------	--------------	------------

Algorithm 3	$O(n)$	$O(n)$	$O(n)$
Algorithm 4	$O(n)$	$O(n)$	$O(n)$

Question: Given the string abcaabdefffgcag

Write an algorithm that verifies that in the string no two adjacent characters are identical.

Algorithm thinking is to look for the number of replacement you will need to do on the string to get the desired result.

Algorithm1 (Counting identical characters)

```
int countReplacements(String s) {
    int count = 0;
    for (int i = 0; i < s.length() - 1; i++) {
        if (s.charAt(i) == s.charAt(i + 1)) {
            count++;
        }
    }
    return count;
}
```

If count is zero, it means that no two adjacent characters are identical

Algorithm1 for the actual replacement of repeated characters.

```
int countReplacementsSim(String s) {
    int count = 0;
    StringBuilder sb = new StringBuilder(s);
    for (int i = 1; i < sb.length(); i++) {
        if (sb.charAt(i) == sb.charAt(i - 1)) {
            count++;
            sb.setCharAt(i, '#'); // simulate replacement
        }
    }
}
```

```
    return count;
}
```

Fill in the cells.

	Best case	Average case	Worst case
Algorithm 1	$O(n)$	$O(n)$	$O(n)$
Algorithm 2	$O(n)$	$O(n)$	$O(n)$

Question: What is the lower bound of the problem?

Solution:

The time lower bound is $\Omega(n)$.

Why — two short proofs / explanations:

1. To be certain that *no* two adjacent characters are equal you must inspect every adjacent pair at least once in the worst case. An adversary can give you a string that is safe up until the last pair, so any algorithm must examine all $n - 1$ pairs to conclude “no adjacent equals.” That gives $\Omega(n)$ time.
2. Counting replacements (optimization): if you must compute the number of replacements needed to eliminate adjacent duplicates, a single missed character could change the count, so you must process the whole string — again $\Omega(n)$.